Smart-Attend Mobile App – SDLC Analysis

Course: SDLC & DevOps Fundamental

Role: Junior Project Manager

1. Introduction

The **Smart-Attend Mobile Application** is a university startup project designed to automate the student attendance process in classrooms. Traditional attendance systems require teachers to manually call student names or circulate attendance sheets, which consumes valuable class time and may lead to errors or manipulation.

The Smart-Attend system aims to solve this problem using **Geo-fencing technology**. Geo-fencing allows the system to detect when a student enters a specific geographic area, such as a classroom, and automatically mark their attendance.

Using a mobile application connected to the university system, students can log in with their university ID. When the system detects that the student has entered the classroom area, the attendance is recorded automatically.

However, during the development process, a major requirement change occurred. The university Dean introduced a new security requirement: the system must include **Biometric Face ID Verification** to prevent students from marking attendance for their friends (known as proxy attendance).

This requirement change occurred halfway through the development process, creating challenges for the development team. As the Junior Project Manager, it is necessary to analyze how the traditional **Waterfall model** would fail in this situation and how switching to **Agile Scrum methodology** can successfully handle the change.

2. Project Scenario

The Smart-Attend system was initially designed with the following core features:

- Student login using university ID
- Geo-fencing to detect classroom entry
- Automatic attendance marking
- Teacher dashboard to monitor attendance

These features allowed the university to automate attendance tracking efficiently.

However, the Dean raised a major concern about proxy attendance. Students could potentially give their phones to friends who enter the classroom, allowing attendance to be marked even if the real student is not present.

To prevent this, the Dean introduced a new requirement:

Biometric Face ID verification must be integrated into the system.

This means that when a student enters the classroom area, the system must verify their identity using facial recognition before marking attendance.

This sudden change significantly impacts the system design and development process.

3. Waterfall Model Failure Analysis

The **Waterfall Model** is a traditional software development methodology where development is divided into sequential phases. Each phase must be completed before moving to the next phase.

Typical Waterfall phases include:

Requirements → System Design → Development → Testing → Deployment

Although this model works well for projects with stable requirements, it becomes problematic when requirements change during development.

In the case of the Smart-Attend system, introducing the Face ID requirement mid-development would create several major issues.

3.1 Rigidity of the Waterfall Model

One of the biggest weaknesses of the Waterfall model is its rigid structure.

Once the requirements phase is completed, the development team moves to the design phase and then to development. If a new requirement is introduced later, it becomes very difficult to go back and modify the earlier stages.

In the Smart-Attend project, the Face ID requirement was introduced **after development had already started**.

To implement this feature, the team would need to revisit several earlier phases:

- Update system requirements
- Modify system architecture
- Redesign authentication mechanisms
- Add camera and biometric processing features

Because the Waterfall model does not support flexible changes, the team would have to **restart several development phases**, causing significant disruption to the project timeline.

3.2 High Cost of Change

Another major problem with the Waterfall model is the **high cost associated with requirement changes**.

In the Waterfall model, the system architecture is built based on the initial requirement specifications. Once the architecture is implemented, making major changes becomes expensive and time-consuming.

Adding Face ID verification requires several new technical components:

- Camera access integration
- Facial recognition algorithms
- Biometric authentication libraries
- Secure storage of biometric data
- Updated database schema

These modifications require additional development effort, security testing, and system integration.

If the project were using Waterfall, implementing these changes mid-development would significantly increase costs and delay the release schedule.

3.3 Delayed Testing and Integration Problems

In the Waterfall model, testing usually occurs near the end of the development cycle.

This means that different system components are integrated and tested only after the entire development phase is completed.

In the Smart-Attend system, the integration of Geo-fencing and Face ID authentication could introduce several potential issues:

- Location detection failures
- Camera permission conflicts
- Biometric authentication errors
- Security vulnerabilities

If these issues are discovered late during testing, the development team would need to redesign and reimplement major system components.

This could delay the project by several weeks or even months.

4. Agile Scrum Solution

To overcome the limitations of the Waterfall model, the Smart-Attend development team decides to switch to the **Agile Scrum framework**.

Agile is an iterative software development approach that focuses on flexibility, continuous improvement, and customer feedback.

Instead of developing the entire system at once, Agile divides development into short cycles called **Sprints**.

Each sprint typically lasts **two weeks** and delivers a working portion of the system.

This approach allows the team to quickly adapt to requirement changes.

5. Sprint Planning

The Smart-Attend project is divided into two major sprints.

Sprint 1 – Minimum Viable Product (MVP)

The goal of Sprint 1 is to develop the basic version of the Smart-Attend system.

Features included in Sprint 1:

- Basic mobile application interface
- Student login using university ID
- Geo-fencing logic for classroom detection
- Automatic attendance marking
- Attendance notification system

At the end of Sprint 1, the development team delivers a working prototype that can automatically mark attendance based on location.

This allows stakeholders to test the system and provide feedback.

Sprint 2 – Feature Enhancement

The goal of Sprint 2 is to improve system security and integrate the new biometric requirement.

Features included in Sprint 2:

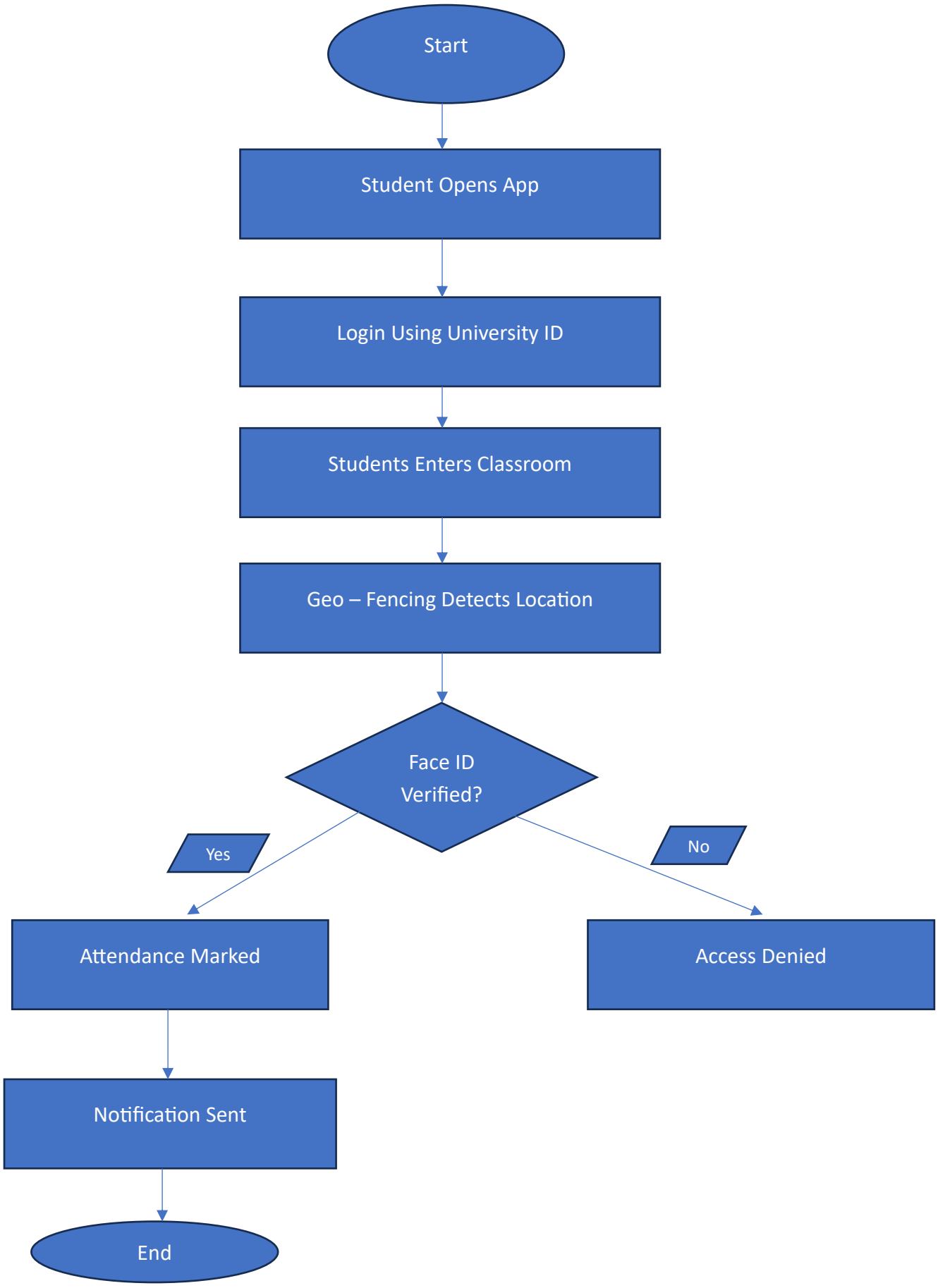
- Face ID biometric verification
- Improved teacher reporting dashboard

- Security improvements
- Performance optimization

By developing this feature in a separate sprint, the team can integrate the biometric system without disrupting the core functionality.

6. User Flow Diagram

The User flow describes how a student interacts with the Smart-Attend System.



Explanation:

1. The student opens the Smart-Attend mobile app.
2. The student logs in using their university credentials.
3. When the student enters the classroom, the geo-fencing system detects their location.
4. The system activates the Face ID verification process.
5. If the biometric verification is successful, the system marks the student as present.
6. The student receives a notification confirming their attendance.

This process ensures that only the actual student can mark their attendance.

7. Benefits of Agile for This Project

Using Agile Scrum provides several advantages for the Smart-Attend project:

- Faster response to requirement changes
- Continuous stakeholder feedback
- Early detection of system issues
- Incremental delivery of features
- Reduced development risk

By implementing Agile practices, the development team can successfully integrate the Face ID requirement without restarting the entire project.

8. Conclusion

The Smart-Attend project demonstrates how traditional development models such as Waterfall struggle to handle changing requirements.

The sudden addition of biometric authentication would cause serious challenges in a Waterfall environment due to rigid development phases, high cost of changes, and delayed testing.

Switching to the Agile Scrum framework allows the development team to adapt quickly and deliver new features through incremental development.

By dividing the project into sprints, the Smart-Attend system can successfully integrate Geo-fencing and Face ID verification while maintaining a stable and efficient development process.